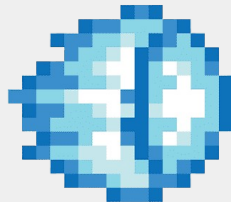
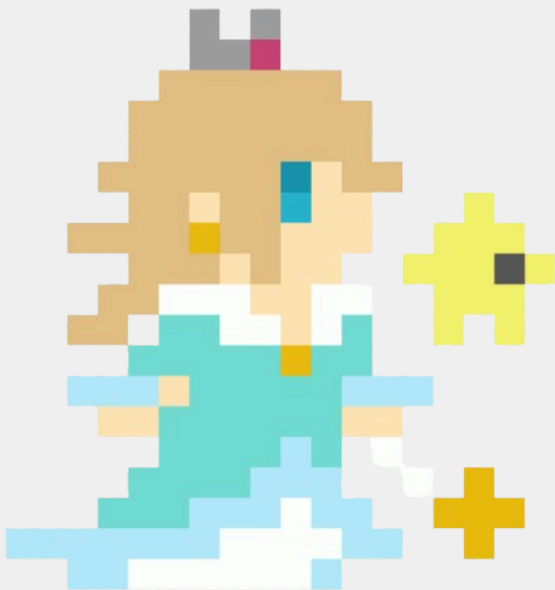


TP DE PROGRAMACION

SUPER ELIZABETH SIS

Universidad Nacional
de General Sarmiento



Integrantes del grupo 6:

Lautaro figueroa: lauti.figueroa.06@gmail.com

Jeremias Pokemayer: jerepoke2016@gmail.com

Ignacio Cendon: ignaciocendon04@gmail.com

Introducción:

"Super Elizabeth Sis" es un videojuego inspirado en el clásico Super Mario Bros, donde los roles se invierten: la Princesa Elizabeth asume el rol de heroína con la misión de rescatar a su plomero Mario, que fue secuestrado por el malvado Rey Camir. A lo largo del escenario, la protagonista debe avanzar superando obstáculos y combatiendo enemigos.

Clase Juego:

La clase juego es la principal coordinadora de todo el juego. Implementa un sistema de estados mediante un switch que determina que pantalla se muestra en cada momento: menu inicial, juego, boss, game over y victoria. Cada estado tiene su propio metodo auxiliar: pantallaInicio(), jugar(), pantallaBoss(), pantallaGameOver() y pantallaWin().

Dentro de jugar() se concentra la mayor parte de la logica, el manejo de la camara actualizando camaraX en funcion de la posicion de la princesa para simular lugar horizontal del mapa, gestiona el spawn de enemigos mediante un temporizador que genera nuevos enemigos de forma alternada desde arriba y desde abajo, coordina todos los loops de actualizaciones: enemigos comunes, enemigos mejorados, proyectiles de la princesa y proyectiles de enemigos. Tambien detecta las colisiones entre los objetos y determina las consecuencias y por ultimo verifica las condiciones de victoria al llegar al castillo o de derrota al quedarse sin vidas.

Clase Princesa:

La clase princesa representa a la protagonista de nuestro juego y se concentra en toda la lógica relacionada con su comportamiento, Al inicializarse recibe su posición en pantalla, comienza con 4 vidas, una velocidad estandar, una velocidad vertical en 0 y en contacto con el suelo.

- **El movimiento horizontal implementamos dos métodos:**

MoverDerecha() y moverIzquierda(). El segundo recibe la posición actual de la cámara como parámetro para que la princesa no pueda retroceder más allá del borde izquierdo visible de la pantalla.

Además en Juego se fuerza una barrera adicional con setX() para evitar que atravesase el límite izquierdo incluso por inercia.

- **La física implementamos actualizarFisica()**

que cada tick suma una constante de gravedad a la velocidad vertical cuando la princesa no está en el suelo, y luego actualiza su posición vertical.

El salto se implementó en saltar(), que solo funciona cuando la princesa está en contacto con una isla, asignándole una velocidad vertical negativa que la impulsa hacia arriba.

- **Para detectar las colisiones de la princesa con las islas se implemento:** `colisionaCon()`, que calcula los bordes de la princesa y de la isla y determina cuatro casos: que la princesa aterrice por arriba, que choque su cabeza por abajo y su colision lateral tanto por izquierda como por derecha. Cada caso reposiciona a la princesa correctamente para que no pueda atravesar ninguna plataforma.

- **En el caso de las vidas de la princesa implementamos** `perderVida()` y `estaViva()`. Cuando la princesa cae al vacío, Juego detecta que su posición vertical, que era cero, superó el alto en pantalla y le resta una vida. si todavía tiene vidas restantes llama a `reiniciar()` que la devuelve a la posición inicial, donde comenzó el juego. Si la princesa llega a cero vidas se crea una nueva instancia de princesa desde 0 y la cámara vuelve al inicio. Exactamente lo mismo sucede cuando un enemigo la toca o su disparo pega en ella.

Un void `perderTresVidas` que disminuye las vidas en 3, utilizada para la colisión con el proyectil del boss.

Las vidas se muestran en pantalla mediante `dibujarVidas()`, que dibuja un icono de corazón por cada vida restante en la esquina izquierda de la pantalla.

- **La misión de la princesa y habilidad del boss:**
Se implemento una misión para la princesa, la cual si la completa recibirá una recompensa, `recuperarVida()`, se activa cuando la princesa elimina a 5 enemigos consecutivos, el contador de eliminaciones se lleva a cabo en Juego y al llegar a 5 llama este método y se le otorga una vida extra a la princesa y reinicia el contador.

Además implementamos el método `perderTresVidas()`, que únicamente se activa en la pantalla del boss y consiste en que el proyectil de este mismo te quitará 3 vidas al impacto, esto para agregarle un poco más de dificultad al juego, las vidas que consigue el jugador con la misión, son acumulables en el boss final.

Clase Proyectil:

Representa la bala que dispara tanto la princesa como los enemigos. Al crearse recibe la posición de la princesa, la posición del cursor del mouse y la cámara actual. Con estos datos calcula la dirección del disparo (Donde apunte el cursor), con estos datos que calcula normaliza el vector dividiendo por la distancia total, Esto garantiza que el proyectil siempre viaje a la misma velocidad sin importar que tan lejos esté el cursor del mismo.

- **Proyectil Enemigo:**
el proyectil tiene un booleano `proyectilEnemigo` que indica de quien proviene el disparo. cuando es false es de la princesa y daña a los enemigos, cuando es true proviene del enemigo y daña a la princesa. Los enemigos tienen su propio sistema de disparo con un timer interno cada cierta cantidad de ticks lanza un proyectil dirigido hacia la posición actual de la princesa, usando la misma lógica de vector que usa la princesa al disparar con el mouse.

- **Cada tick actualizar()**

suma velocidad en X e Y a la posición del proyectil, moviendolo en línea recta hacia donde indique el cursor.

- **La deteccion de colision**

con los enemigos se implemento en colisionaCon(), que calcula la distancia entre el centro del proyectil y el centro del enemigo. Si esa distancia es menor que la suma del radio del proyectil y la mitad del ancho del enemigo, hay colisión.

En juego se maneja solo que pueda existir un solo proyectil a la vez: mientras el proyectil no sea null, no se puede disparar de nuevo. Al colisionar con un enemigo ambos se vuelven null, y si sale de los límites de la pantalla sin tocar nada, también se vuelve null.

Clase Enemigos:

En la clase Enemigos Junto con la clase paralela a esta EnemigosMejorados, hicimos el comportamiento básico de los npc, el cómo se representan visualmente a la hora de ejecutar el programa e interacciones físicas con los otros objetos que se mostrarán por pantalla

X e Y: definimos coordenadas en la matriz del mapa para poder mover a los npc y por donde spawnearan. ANCHO y ALTO: Definimos dimensiones del área que funciona como colisión y para darle forma al enemigo. DIRECCIÓN: variable de movimiento, para forzar el movimiento lateral hacia la izquierda de los npc.

VELOCIDAD: variable que define el ritmo de desplazamiento del enemigo a través del mapa

- **Colisiones:**

Public colisionConPrincesa: El método de colisión sirve para saber si el enemigo chocó con la princesa. Utilizamos las variables definidas “ancho”, “alto” para saber las dimensiones exactas, el código calcula constantemente dónde están los bordes de cada objeto (arriba,abajo,izquierda, etc) si se chocan, el método devuelve true para avisarle al juego que hubo una interacción.

- **Eliminar los arreglos objetos creados:**

Public salioPantalla: para evitar que el juego se sobrecargue con enemigos infinitos al, se implementa esta función que funciona con camaraX, si un npc supera el límite fuera de la pantalla de la camaraX el método le avisa al código central para que vuelva a los arreglos objetos creados en null.

- **Implementación de “aparicion”**

El sistema de generación de enemigos funciona mediante un temporizador en la clase Juego que controla su frecuencia de aparición. Cada vez que este contador

llega a su límite, se invoca alternadamente a los métodos estáticos crearArriba() o crearAbajo() de la clase Enemigos para generarlos en ubicaciones fijas

- **Enemigos Mejorados:**

Se implemento una nueva clase que representa el segundo tipo de enemigo del juego llamada EnemigosMejorados, a diferencia de los enemigos normales, estos tienen 2 vidas, lo que significa que van a requerir 2 disparos para poder derrotarlos, su velocidad es mayor haciendolos considerablemente mas poderosos, estan ubicados en las mismas posiciones que los enemigos comunes pero su spawn tiene menos frecuencia.

Clase Castillo:

Tiene variables privadas x, y, ancho, alto e imagen, un constructor para las variables, con getters y setters de X e Y.

- Un void **dibujarCastillo**

que toma como parámetro entorno y camaraX, dibuja el castillo en pantalla.

- Cree una función **colisionConPrincesa**

que toma como parámetro a princesa, detecta cuando la princesa llega a la puerta del castillo.

Clase Isla:

Tiene variables privadas x, y, ancho, alto e Imagen, un constructor para dichas variables, un void para dibujar la isla que toma como parametro entorno y la posicion de camaraX, getters y setters de cada variable.

- Para dibujarlo

en la clase Juego se genera un arreglo de isla, las primeras son islas base con tamaño fijo ya que contienen los pozos a los que, si la princesa cae pierde una vida y los demás son las islas flotantes, con posiciones y anchos random. La variable islaBoss pertenece a la isla base de pantallaBoss.

Al dibujarlo se superponían entonces cree una función boolean alejaDe, que detecte cuando colisionen alguno de los cuatro costados del rectángulo y tengan un margen tanto de ancho como de alto para generarse, con un ciclo **for** recorro el arreglo de islas antes de guardarlas en el mismo, genero la isla nueva, ingreso al ciclo **while** que contiene un **for** adentro que con un **if** verifica si nueva colisiona con la isla en posición j ya guardada, si colisiona en nueva se guarda otra isla con parámetros distintos.

Como las islas tienen que estar en todo el mapa, en lo que no se ve y en lo que se ve, en la clase Juego cree el **int** `camaraX` que fija en que parte del mapa esta la pantalla, se cambia la visión en el eje X con relación al movimiento del personaje.

Clase Boss:

Tiene variables privadas `x`, `y`, `ancho`, `alto`, `vida`, `imagen` e `imagenProyectil`, con un constructor para dichas variables.

Un void **dibujar** para mostrar en pantalla el boss y el proyectil.

Un void **disparar** que toma como parámetro la princesa, utiliza sus getters X e Y y direcciona el proyectil hacia su posición.

Un boolean **colisionaCon** que toma como parámetro al proyectil, verifica si el proyectil de princesa colisiona con el boss. Un **getter y setter** de vida para modificarlo cuando los proyectiles de la princesa colisionen. La vida se setea y dibuja con un rectangle por cada vida y a medida que colisionan los proyectiles de princesa se van restando de a 1.

Un void **actualizar** que toma como parámetro a princesa y entorno, verifica si el proyectil está activo, si no activa nuevamente la opción de disparo del boss, si sale de la pantalla se elimina.

Un void **actualizarProyectil** que si el proyectil está activo, cambia la velocidad.

Un boolean **proyectilGolpea** que toma como parámetro a princesa, verifica si el proyectil del boss golpea a la princesa mediante la fórmula de distancia entre puntos.

Un boolean **proyectilSalio** que toma como parámetro el entorno y verifica si el proyectil sale de pantalla.

Problemas encontrados/soluciones:

Problema: que se manifestó fue de como hacer un random de apariciones de enemigos que no estén fijados sino que aparecieran infinitamente en el mapa.

Solución: Decidimos descartar el uso de *random* y diseñar una solución fija basada en métodos con un eje Y y eje X `crearArriba` y `crearAbajo` que toman la posición de la cámara y colocan a los enemigos en coordenadas de altura fijas con esto los enemigos aparecerán arriba y abajo intercalandose.

Problema: Las islas colisionaban entre ellas al momento de generarse.

Solución: función boolean `alejaDe` que detecta la colision y con un arreglo se van recorriendo y si hay que cambiar la isla porque `alejaDe` es true se genera otra y se cambia.

Problema: las imágenes de isla quedaban algunas mas grandes y otras mas chicas

Solución: utilice `getScaledInstance` que redimensiona la imagen al ancho y alto deseado.

Problema: Sincronizar la posición real de los objetos en el mundo con lo que se muestra en pantalla.

Solución: mantener la coordenadas reales de cada objeto en el mundo y restarle `camaraX` unicamente al momento de dibujar. De esta forma la princesa siempre aparece centrada visualmente mientras su posición sigue avanzando en el mapa.

Problema: Al implementar el disparo tanto de la princesa como de los enemigos, surgió el problema de que la velocidad del proyectil variaba según la distancia del objetivo

Solución: Normalizar el vector de dirección dividiendo cada componente por la distancia total, obteniendo así un vector de módulo 1 al que luego se le multiplica el valor de la velocidad deseada, garantizando que el proyectil viaje siempre a la misma velocidad independientemente de la distancia.

CONCLUSIONES:

El desarrollo de “Super Elizabeth Sis” nos permitió aplicar en práctica los conceptos fundamentales aprendidos durante toda la cursada. Ya que partimos de un proyecto limpio en el cual tuvimos que empezar casi de cero creando todas las clases que necesitábamos para que el juego funcione de la manera que queríamos.

Uno de las cosas más importantes fue comprender el valor de la organización del código, ya que si este mismo estaba desordenado o no se entendía, no se podía trabajar en equipo de una manera correcta, además de que uno de los requisitos del trabajo era que el código este prolijo y sea legible, aunque al principio lo más fácil era poner toda la lógica en Juego, pero a medida que avanzábamos en el proyecto, quedó claro que lo mejor era separarlo en varias clases para poder facilitarnos el modificar cada clase de una manera mucho más rápida y eficaz, además de que cuando había un error en una clase, sabíamos donde y como era, con lo cual podíamos encontrar mucho más rápido la solución al mismo.

También, al ser la primera vez que utilizábamos la aplicación Eclipse para programar, mas siendo un proyecto grande como lo es el juego, podía generar dudas de funcionamiento de la misma aplicación que con la experiencia utilizándola varios días se fueron resolviendo solos, además aprendimos que los errores más difíciles de solucionar no son los de sintaxis, que te marca eclipse, si no los de lógica, como los que están remarcados en la categoría **PROBLEMAS**.

En definitiva el trabajo no solo nos dejó conocimientos técnicos sobre la programación orientada a objetos, si no también experiencia en construir un proyecto de software real en equipo, con todos los problemas y decisiones que eso implica.